从逻辑回归到 Agentic Programming

贷款违约预测实验课 · 数据、模型与可验证的 Agent 工作流

01 DATA

02 MODEL

03 AGENT

模型线 × 工程线

2026 · 秋季

这门课有两条主线

模型线 / MODEL

从一张表到可解释的概率

01 数据与清洗
47 列 → 44 维；缺失值、日期、类别编码

02 逻辑回归
logit → sigmoid → BCE； L1/L2 正则化

03 训练
Dataset / DataLoader； AdamW 100 epochs

04 评估
混淆矩阵、 Precision/Recall 、 ROC-AUC

工程线 / AGENT

从一句任务到可验收的交付

01 读任务
手册、评分细则、数据与代码边界

02 规划
把目标拆成可单独验证的小步

03 行动
编辑、运行、看报错、再迭代

04 验收
测试、指标、日志与人工复核

带走四件事：解释 • 实现 • 辨别 • 交付

两条线在同一个实验里汇合：用 Agent 完成一个可验证的逻辑回归实验。

从回归到分类：为什么要多一步？

回归

预测一个连续值

房价、销量、温度

输出 2.7 有数量含义

分类

预测一个类别 / 概率

违约 / 不违约

输出 1.7 没有概率含义

训练集 用来拟合参数的样本集合

样本 一条贷款记录， $x^{(i)}$

标签 要预测的目标 $y^{(i)}$ = isDefault

特征 用来预测的输入列

逻辑回归的做法：
先算线性分数（logit），
再用 sigmoid 变成概率。

如果硬用线性回归：输出可能是 -0.3 或 1.7，无法当概率。

所以它不是“回归”房价，而是用回归的分数做二分类。

一笔贷款，要不要批？

某信贷平台的 10,000 条贷款记录。每条记录 47 个字段，标签是 isDefault： 1 = 违约。

- 47 个申请人字段：金额、期限、利率、收入、信用、就业……
- 10,000 条历史记录；训练 8000 / 验证 1000 / 测试 1000
- 标签 50.06% 为正样本——接近均衡， accuracy 有参考价值
- 代价不对称：漏掉违约，和误拒好客户，是两种错误

为什么用概率而不是硬标签？概率保留不确定性；阈值可以按业务代价调整。

47

输入字段

含 15 列匿名行为计数特征

10,000

历史贷款

训练 / 验证 / 测试 = 8:1:1

50.06%

训练集正样本

接近 1:1 ，类别均衡

模型输出概率；阈值输出决策。

数据全景： 47 列，最后只留 44 维

47

原始列

id / 标签 / 特征

44

输入特征

去掉 id 、标签、 subGrade

8k / 1k / 1k

train / val / test

共 10,000 条

50.06%

训练集正样本

4005 / 8000

数据划分



42 数值 *

33 连续 + 9 离散

金额、利率、收入、工作年限计数……

5 类别 *

grade / employmentLength / 时间

清洗后转成数字，不丢语义

15 匿名

n0 – n14

申请人行为计数；含义未标注

注：任务 PPT 说 “ 2000 测试”，实际文件拆成 val 1000 + test 1000； * 为数据字典口径，精确列账本见下页。

列账本： 47 列怎么变成 44 维

47	− 1	− 1	− 1	44
原始列	id	isDefault （标签）	subGrade	输入特征

44 = 40 个原始数值列 + 4 个编码列（ grade / employmentLength / issueDate / earliesCreditLine ）

40 原始数值

33 连续 + 9 离散，减去 id / 标签 / subGrade 后剩下

4 编码列

grade A=1…G=7； employmentLength 1/n+1/12；两个日期转成天数 / 月数

1 个常量

policyCode 只有 1 个值；保留但在 MinMax 后恒为 0

任务 PPT 的“42 数值 + 5 类别 + 15 匿名”是数据字典的近似说法，三组数字对不上；以这张列账本为准，学生自检写 train_input.size(1) == 44 。

环境：三条命令跑起来

手册 §1.5.1 的最小环境

```
conda create -n logic python=3.9 -y
conda activate logic
pip install torch==2.6.0 numpy==1.26.4 \
    pandas scikit-learn jupyter tqdm
```

- Python 3.9+ 、 PyTorch 、 Jupyter ； 版本不严格，能 import 即可
- VSCode 选择 logic / .venv 解释器，再打开 logic.ipynb
- 先跑一个 import torch 和 print(torch.__version__)

01 先隔离

虚拟环境避免依赖冲突；本次使用 Python 3.11 + torch 2.14 CPU 验证。

02 再验证

import torch 、读一个 CSV 、打印 shape—— 三步确认环境真的可用。

03 后写模型

把“跑不通”和“模型不对”分开，排错会快很多。

环境不是评分重点，但没有它，后面所有指标都无从谈起。

清洗：把语义变成数字

原始列	处理	结果
id	唯一标识，与违约无关	删除
issueDate	yyyy-mm-dd	距最早日期的天数
earliesCreditLine	MMM-YYYY，如 Mar-2007	距最早月份的月数
grade / subGrade	等级 A–G 与子等级	删 subGrade； grade A=1 … G=7
employmentLength	“< 1 year” “3 years” “10+ years”	1 / n+1 / 12

全部封装在 utils.py::data_utils.pipeline()；本实验不改 utils.py。

转换完再 astype(float)，得到纯数值表；这一步决定了后面 44 维的形状。

缺失值：标签不能猜，特征可以填

先删标签缺失，再填特征均值

```
def filter_NA(df):
    return df.dropna(subset=["isDefault"])

def fill_mean(df):
    return df.fillna(df.mean(numeric_only=True))

train, val, test = filter_NA(train), ...
train, val, test = fill_mean(train), ...
```

训练集 5595 个缺失单元， 19 列有缺失； n11 624 、 employmentLength 518 、 n0–n14 多数 325 。

标签

isDefault 为 NaN → 删除
标签无法从特征恢复；不能猜。

特征

用同列均值填充
简单可解释的基线；中位数 / 模型填补也可以。

验收

填充后 0 个 NaN
三份数据都要检查，不能只看 train 。

从表到张量：先对齐形状

DataFrame
8000 × 44
float64

→

MinMaxScaler
只在 train 上 fit
再 transform val/test

→

Tensor
X: (8000, 44)
float32

→

Label
y: (8000,)
0 / 1

三份数据的形状

X_train	(8000, 44)	X_val	(1000, 44)	X_test	(1000, 44)
y_train	(8000,)	y_val	(1000,)	y_test	(1000,)

为什么缩放：年收入 / 余额与利率的量级相差 3–4 个数量级； MinMax 让梯度更稳定。

易错点： MinMaxScaler 只在 train 上 fit ；否则测试集信息会泄漏到训练。

标签保持 1-D ， 模型输出 2-D ； 后面会在 BCELoss 前统一 flatten 。

MyDataset： 两个魔术方法

数据集只负责“一条样本”

```
class MyDataset(Dataset):
    def __init__(self, *tensors):
        assert all(t.size(0) == tensors[0].size(0)
                   for t in tensors)
        self.tensor = tensors

    def __len__(self):
        return self.tensor[0].size(0)

    def __getitem__(self, index):
        return tuple(t[index] for t in self.tensor)
```

assert 各张量首维一致：形状错误应该在构造 Dataset 时就暴露，而不是训练到一半。

Dataset 负责“一条”， DataLoader 负责“一批”。

__len__

返回 8000
DataLoader 用它算 batch 数

__getitem__

返回 (x_i, y_i)
x: (44,), y: 标量

谁负责批

DataLoader
负责 batch / shuffle

DataLoader：把 8000 行切成 125 个 batch

batch=64；只打乱 train

```
train_loader = DataLoader(
    MyDataset(X_train, y_train),
    batch_size=64, shuffle=True)

val_loader = DataLoader(
    MyDataset(X_val, y_val),
    batch_size=64, shuffle=False)
```

- 训练： 125 个 batch / epoch （8000 ÷ 64，向上取整）
- 验证 / 测试： 16 个 batch / epoch （1000 ÷ 64，末批 40）
- shuffle=False 让评估顺序固定，方便复现
- 训练 shuffle 用独立 generator，避免验证集迭代消耗全局 RNG

64

每批样本

显存 / 内存与梯度噪声的折中

125

train batch

一个 epoch 更新 125 次

16 / 16

val / test

只做前向，不打乱

02

从分数到概率

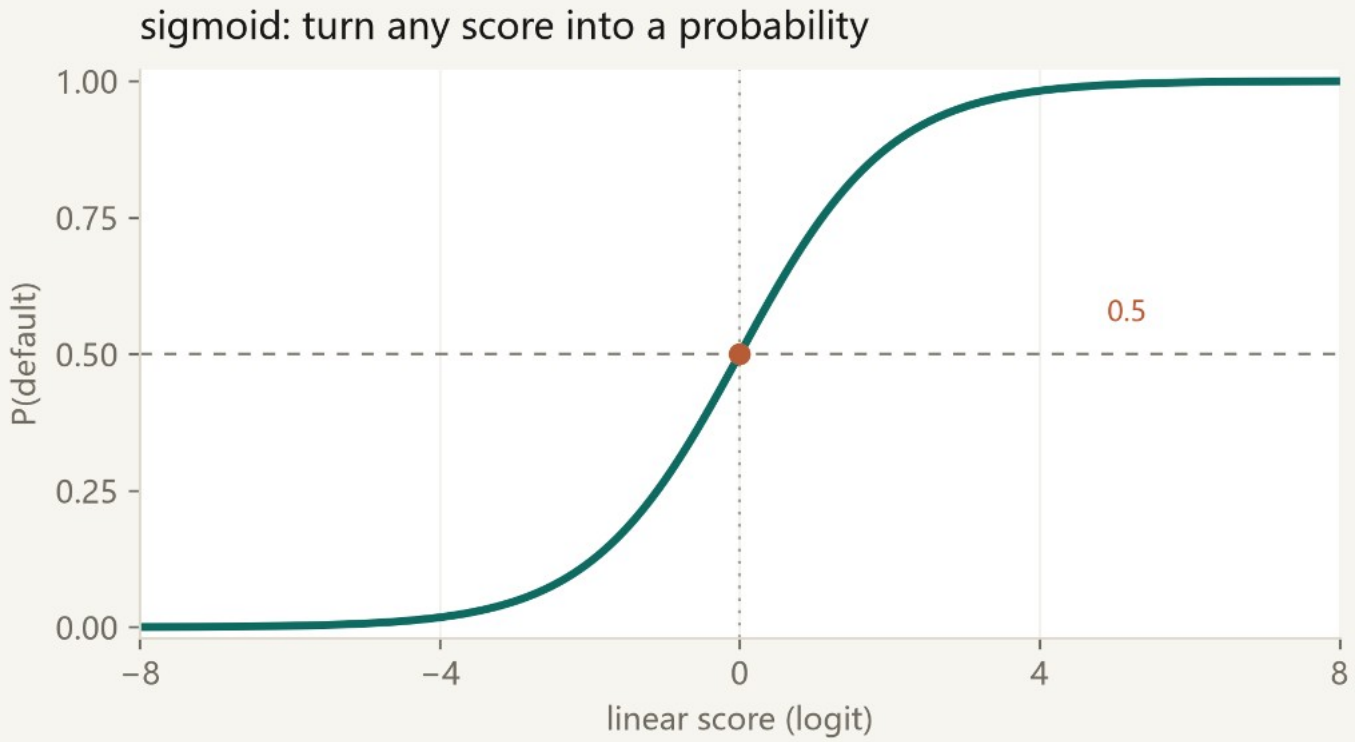
logit → sigmoid → BCE → 正则化

逻辑回归 = 线性分数 + sigmoid

$$z = \mathbf{w}^\top \mathbf{x} + b$$

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- 线性分数没有边界：(-∞, +∞)
- sigmoid 把它压到 (0, 1)，变成概率
- z 为 0 时概率 0.5； ±2 时约 0.88 / 0.12
- p 是“违约概率”，不是硬标签



为什么是交叉熵?

把每个样本看成一次伯努利试验:

$$P(y=1 \mid x)=p, \quad P(y=0 \mid x)=1-p$$

$$L=\Pi_i p_i^{y_i}(1-p_i)^{1-y_i}$$

$$\mathrm{BCE}=-[y\log p+(1-y)\log(1-p)]$$

最大化似然 = 最小化负对数似然。对数把连乘变成连加，也让“自信且错误”的惩罚更大。

1 写似然 → 2 取对数 → 3 加负号 = BCE

真实 y	预测 p	BCE
1	0.90	0.105
1	0.10	2.303
0	0.10	0.105
0	0.90	2.303

注意： p=0.10 时，真实 y=1 的损失是 2.303 。

BCE 对“自信且错误”的惩罚急剧增大。

逻辑回归的损失不是随便选的：它就是 sigmoid + 伯努利分布的负对数似然。

正则化：别把噪声当规律

L1

$$R_{L1}(w) = \lambda \Sigma_j |w_j|$$

稀疏：很多权重变成 0

适用：特征多、想要可解释时

本实验： 34/44 个权重 ≈ 0

L2

$$R_{L2}(w) = \lambda \Sigma_j w_j^2$$

收缩：权重变小但不为 0

适用：想要更平滑的解时

本实验：权重整体缩小，但不为 0

本实验： L1 + norm_weight = 0.005

L1 范数从 31.5 降到 3.61； 34 个权重≈ 0 。测试 AUC 与 L2/ 无正则接近（ 0.6897 / 0.6914 / 0.6921 ）—— L1 的主要收益是稀疏与可解释，不是更高的分数。

模型实现：一个类，三个关键点

模型 + 损失（节选）

```
class LogisticRegression(nn.Module):
    def __init__(self, d, norm="l1"):
        self.w = nn.Parameter(torch.randn(d, 1))
        self.b = nn.Parameter(torch.randn(1))

    def l1_norm(self):
        return torch.abs(self.w).sum()

    def l2_norm(self):
        return (self.w ** 2).sum()

    def forward(self, x):
        return torch.sigmoid(x @ self.w + self.b)

def criterion(p, y):
    p = torch.clamp(p.view(-1), 1e-7, 1 - 1e-7)
    return nn.BCELoss()(p, y.view(-1))
```

参数只有 45 个；模型部分对应 4 个 TODO： init / l1_norm / l2_norm / sigmoid / forward / criterion 。

形状

p: (N, 1)
y: (N,)
→ 先 flatten

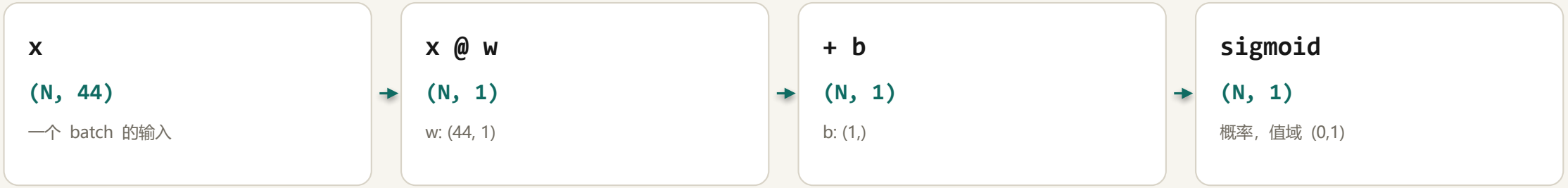
数值

sigmoid 饱和
→ clamp
防 log(0)

正则

normnize()
L1 / L2
一行切换

forward： 形状只变一次



参数只有 44 × 1 + 1 = 45 个；模型很小，瓶颈在特征，不在容量。

为什么后面要 flatten

BCELoss 期望 (N,) 的概率和标签；模型输出 (N,1)，所以在 criterion 里 view(-1)。

常见错误

直接 BCELoss(pred, y) 会把 (N,1) 与 (N,) 广播成 (N,N)； loss 仍然是一个数，但含义已经错了。

训练配置：一屏看懂

项目	值	说明
optimizer	AdamW	自适应学习率 + 解耦权重衰减
lr	0.005	手册配置；不要一上来调太大
epochs	100	本数据约 10 秒跑完
batch_size	64	125 个 train batch / epoch
正则	L1, 0.005	norm_weight ；加在 BCE 上
阈值	0.5	概率 → 0/1 的决策边界
seed	42	参数初始化 + DataLoader

配置不是越复杂越好：先把 5 个 epoch 跑通，再逐项改；每次只改一个变量。

AdamW 的默认值

weight_decay=0.01 是模板默认，等于额外 L2 衰减；本实验没有关掉它。

学习率 0.005

太大震荡，太小收敛慢；先跑 5 个 epoch 看 loss 是否下降。

100 个 epoch

在这个数据上 10 秒左右；先跑通，再谈早停与调参。

训练循环：五步 + 一次验证

1. **zero_grad** 清空上一轮梯度
2. **forward** p = model(xb)
3. **loss** BCE + 正则项
4. **backward** 自动求梯度
5. **step** AdamW 更新参数

每个 epoch 后：model.eval() + no_grad() 跑验证集。

内层循环：六行

```
for xb, yb in train_loader:
    opt.zero_grad()
    p = model(xb)
    loss = criterion(p, yb) + 0.005 * model.normnize()
    loss.backward()
    opt.step()
```

模板里的三个小问题

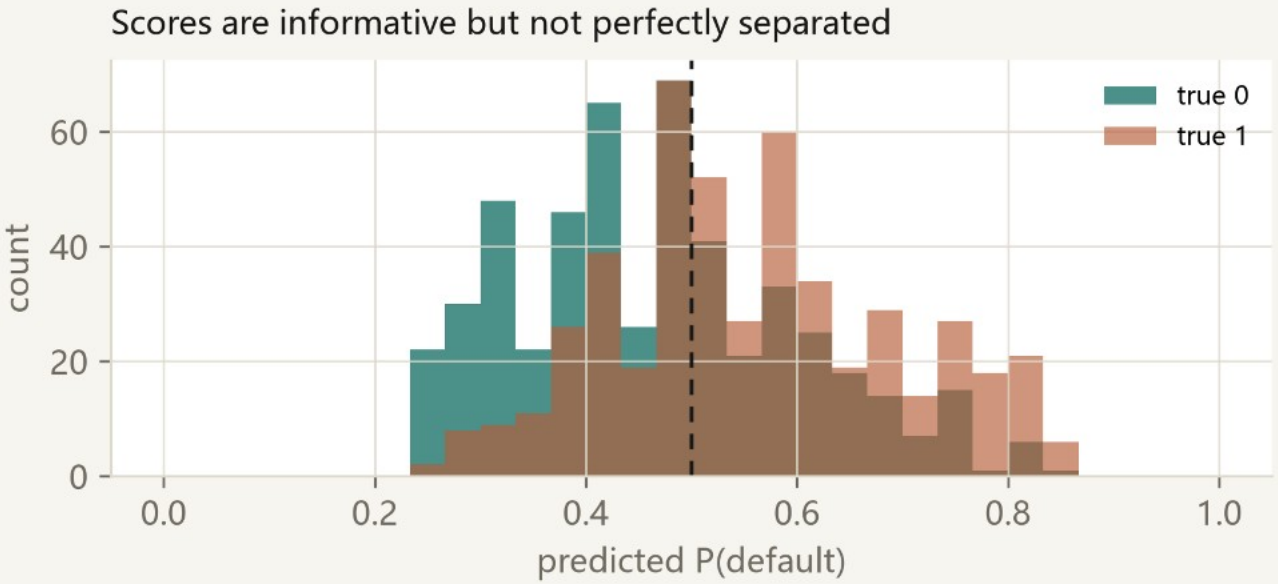
① 先验证后训练；② val_input/val_label 覆盖外层变量；③ 每个 batch 新建 loss_fn 。建议改成先训练后验证，变量名用 xb/yb 。

评估：别被 50% 骗了

	Pred 0	Pred 1
True 0	328	182
True 1	183	307

测试集，阈值 = 0.5

Accuracy	**0.635**	Precision	**0.628**
Recall	**0.627**	F1	**0.627**
ROC-AUC	**0.690**		

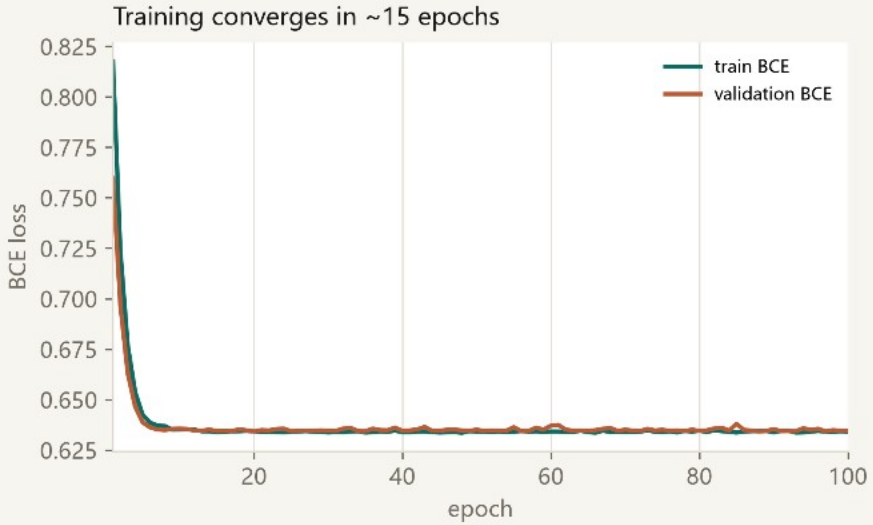


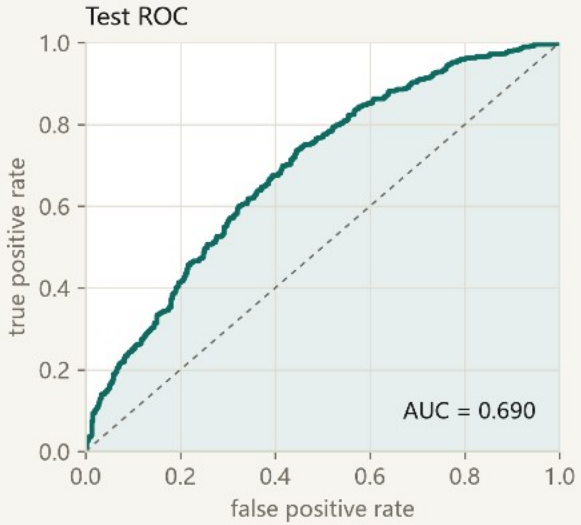
AUC 与阈值无关；阈值升高通常 precision↑、recall↓。

本数据接近 1:1，所以 accuracy 有参考价值；类别不均衡时不能只看它。

结果： 0.635 / 0.690

数据	Accuracy	Precision	Recall	F1	ROC-AUC
train (8000)	0.6446	0.6453	0.6442	0.6448	0.7005
val (1000)	0.6550	0.6606	0.6515	0.6560	0.6969
test (1000)	0.6350	0.6278	0.6265	0.6272	0.6897





与手册对齐

Val 0.6550

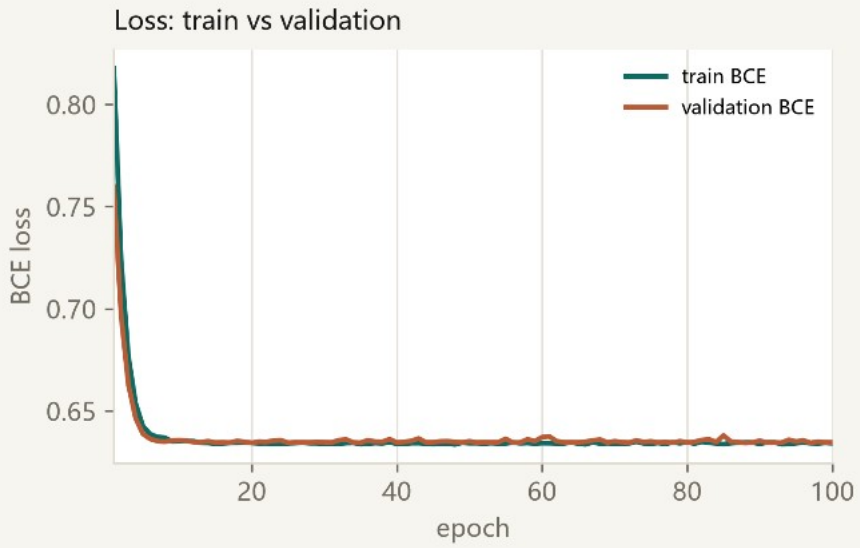
手册示例是 0.655； 数字吻合说明管线与配置符合预期，但不是正确性证明。

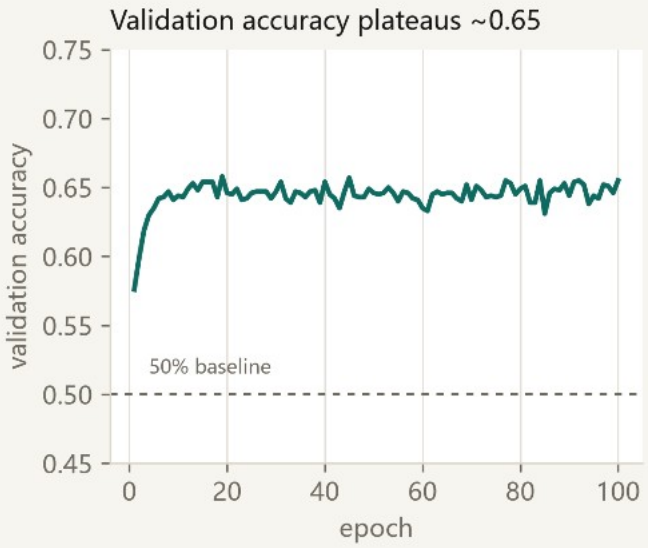
Test AUC 0.6897

L1 范数 3.61

50% 是基线； 1000 条测试集标准误约 1.5pt； sklearn 线性基线 AUC 0.692 、 HGB 0.695——0.69 已接近这批特征的上限，不要过度解读 1% 差异。

曲线在说什么：过拟合、欠拟合还是瓶颈?





过拟合

train 持续下降， val 先降后升， gap 变大。

欠拟合

train 和 val 都高、都不降；模型太弱或没训够。

信息瓶颈

train ≈ val ，都不高；特征本身信号有限。

本实验 train ≈ val ≈ 0.65：更像“信息瓶颈”，不是过拟合；所以加参数、加层通常没用。

六个高频坑：症状 → 修法

01 BCELoss 广播

p (N,1) + y (N,) → (N,N) 。 loss 看似正常但不收敛。修：view(-1) 。

02 log(0)

sigmoid 饱和到 0/1 ， 出现 NaN/Inf 。修： clamp 到 [1e-7, 1-1e-7] 。

03 RNG 耦合

验证集迭代也消耗 RNG ； notebook 与脚本不同。修：独立 generator 。

04 L1 过稀疏

44 维、 0.005 偏强； 34 个权重≈ 0 。先看验证曲线，再调权重。

05 dtype 不一致

标签用 long 、概率用 float64 。 BCELoss 报错。修：显式 float32 。

06 模板顺序 / 遮蔽

先验证后训练、 val_* 覆盖外层变量。改成先训练后验证，用 xb/yb 。

能复现，才算定位；能验收，才算修好。

评分与提交： 10 分，两个硬要求

评分块	分值	交付证据
环境与数据	3	环境、读取、清洗、 Dataset/DataLoader
模型	4	init 、 sigmoid 、 forward 、 L1/L2
训练与评估	3	BCE 、测试评估、最后 markdown 反思
合计	10	

提交清单

- 可执行的 logic.ipynb
- 最后一格 markdown 反思
- 学号 _ 姓名 _logic.zip
- verify_submission.py 全 PASS

代码对照：清洗 c3/c5 · Dataset c9/c11 · 模型 c14 · 评估 c16 · 反思末格

手册示例 Val Accuracy = 0.655 ； 本次 0.6550 。

数字吻合不是正确性证明；评分看的是可运行、可复核的证据。

03

从模型到 Agentic Programming

Agent · Prompt · Context · Harness

Agent 比 LLM 老得多

Agent = 能感知环境、作出决策、采取行动影响环境的实体。

感知

`sensors`

读文件、 grep 、终端输出、测试报错

→

决策

`policy / LLM`

选下一步：读、改、运行、问人

→

行动

`actuators`

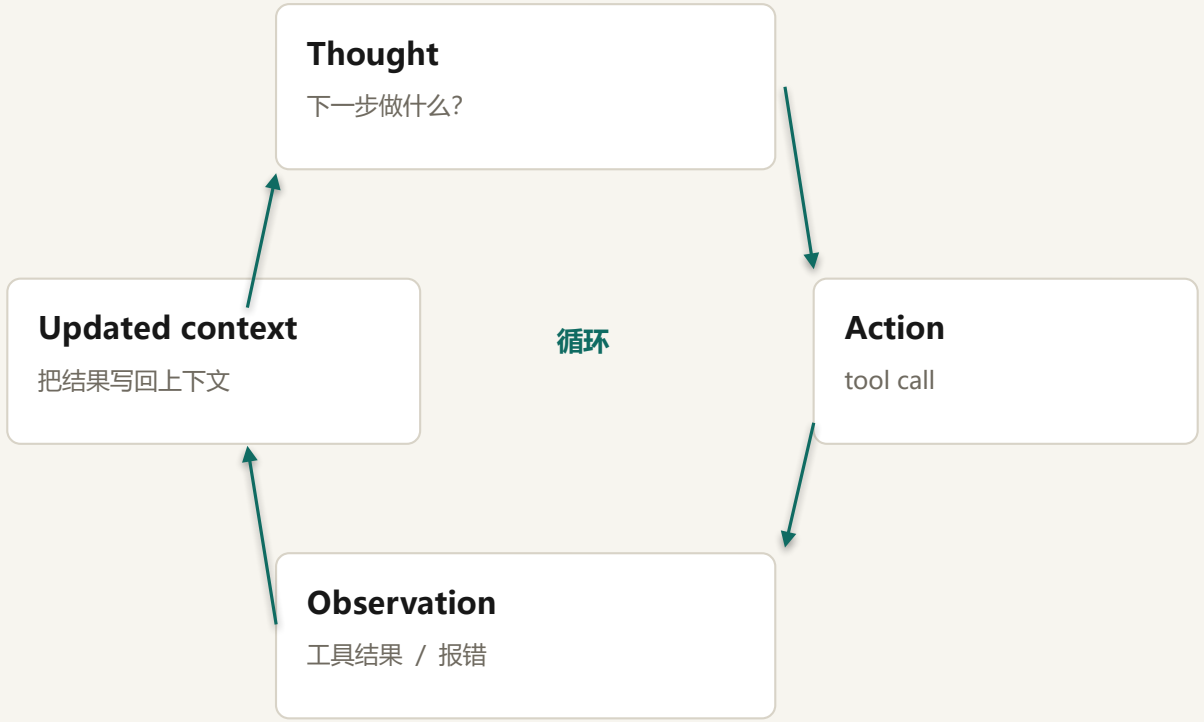
编辑文件、跑 python 、调用工具

概念时间线

1995	2000s	2022	2024	2025–26
Wooldridge & Jennings	**AIMA 第 2 章**	**ReAct**	**Coding agents**	**Harness engineering**
自主、交互、感知、主动	感知—决策—行动	推理与行动交错	工具 + 测试 + 仓库	接口、权限、验收

这四条性质不要求神经网络，也不要求自然语言。 LLM 只是换了一个决策核心。

LLM Agent：想一步，做一步，看一步



- 模型只输出 tool call （工具名 + 参数）
- Harness 执行工具，把结果回填上下文
- 环境反馈是 ground truth：测试、报错、指标
- 要设停止条件：最大轮数、预算、人工 checkpoint

Coding agent 的例子

读文件 → 跑测试 → 看报错 → 改代码 → 再跑。

Agent 不等于 Workflow

	Workflow	Agent
路径	代码预先写死	模型动态决定
适合	稳定、可预测的任务	开放、步骤未知的任务
代价	低延迟、可控	更贵；错误可能累积
例子	ETL 、报表、固定流程	修 bug 、研究、多文件改动

先找最简单的方案；只有确有必要时才加复杂度。

Anthropic： workflow 由预定义代码路径编排； agent 由模型动态决定过程与工具使用。

同一个模型，三种 Harness

	A • 纯聊天	B • 有工具，无验收	C • 完整 Harness
能看到	你粘贴的一段文本	整个工作区	工作区 + 手册 + 历史 + 报错
能做什么	只能回答	改文件、跑命令	同 B，且有权限边界与可追溯记录
记得什么	当前对话	当前会话	日志 + 项目文件 + 跨轮目标
怎么算完成	你说了算	它自己说完成	测试、自检、日志三方对上

可完成的工作 ≈ 模型能力 × 观察带宽 × 动作带宽 × 反馈质量 × 状态连续性

这是教学近似，不是定律；但它解释了“为什么同一个模型在不同工具里表现差很多”。

Prompt： 把任务说到可执行

角色	你是谁：课程助教 / 审稿人 / 数据工程师
任务	一个可验收的目标：补全并跑通 notebook
上下文	手册、数据、 utils.py 、报错片段
约束	不改划分、不调测试集、 seed=42
输出	改动摘要 + 命令 + 指标表

Prompt 的局限：拿不到新信息、不能保证执行、不能自证正确。

一个够用的 prompt 骨架

```
角色：课程助教，熟悉 PyTorch 与手册 §1.6 。
任务：补全 notebook 的 TODO ，使其从头到尾可执行。
上下文：手册在 source_materials/ ，数据在 logreg_lab/data/ 。
约束：不改数据划分；不调测试集； seed 固定 42 。
输出：改动摘要 + 复现命令 + Val/Test 指标。
自检：运行 verify_submission.py ，报告失败项。
```

few-shot | **任务分解** | **显式约束**

结构化输出 | **先复述验收标准**

Context： 有限的注意力预算

Context = 一次采样中模型能看到的全部 token； 它不是越多越好。

- Context rot： 上下文越长，准确召回越弱
- 好的 context 是“最小的高信号 token 集合”
- 渐进披露：先给索引，需要时再取细节
- 长任务：压缩、结构化笔记、子 agent

一次请求的上下文预算



太多 → 关键信号被淹没；太少 → 模型没有依据。

Compaction 把长历史压缩成摘要，保留决策与未解决问题。

Notes 把进度写到文件 / to-do，跨轮恢复。

Sub-agents 用独立上下文做深搜，只回传摘要。

给模型正确的信息，而不是更多信息。

Harness： 让循环真正跑起来

模型提出行动； Harness 让行动发生，并把结果变成下一步的输入。

01	**Agent Loop**	组织模型调用 → 工具执行 → 结果回填
02	**Context Management**	决定每一步放进窗口的内容
03	**Tool Runtime**	读写文件、跑 python 、 grep 、后台任务
04	**Sandbox & Permissions**	限制可写范围，管理授权与拒绝
05	**Session & Memory**	会话状态、日志、跨轮连续性
06	**Orchestration**	子任务、子 agent 、多阶段协作
07	**Logging & Evaluation**	记录过程，提供测试、指标与验收

Harness 不是“更长的 prompt”；它是让 agent 能看、能做、能记、能验收的软件系统。

同一个模型，不同的 Harness

Prompt **怎么说**

任务、角色、约束、输出格式

Context **看到什么**

文件、工具输出、历史、记忆

Harness **能做什么 / 怎么算完成**

工具、权限、循环、日志、验收

AGENTS.md / GOAL.md

文件是上下文与指令的载体，不是发动机。 Harness 决定何时读、读了算不算数；DSH 的 /goal 是运行时状态。

验收属于 Harness

测试、指标、日志、人工复核——没有这些，“完成了”只是模型的一句话。

ACI：为 agent 设计的工具接口，会直接影响它的表现。

模型决定上限， Harness 决定下限，验收决定我们凭什么说“做完了”。

用 Agent 完成本实验：一条可验收的循环

01	**读任务**	手册 §1.6 、数据、 utils.py 、已有 notebook
02	**计划**	拆成数据 / 模型 / 训练 / 评估 / 反思
03	**看现场**	shape 、缺失值、标签分布、基线
04	**改**	只改必要行；保持可执行
05	**跑**	从头执行；看报错和日志
06	**验**	43 项自检 + 指标口径 + 人工复核
07	**记**	worklog ： 命令、数据、问题、结论

可直接改写的任务书

```
任务：补全并验证这次逻辑回归实验。
输入：手册 §1.6 、 data/ 、 utils.py 、 Logic.ipynb 。
约束：不改划分；不调测试集； seed=42 ；
   每项指标必须能在 outputs/ 复核。
验收： verify_submission.py 全 PASS ，
   并在最后写一段自己的反思。
```

人工检查点

- 方法学：均值填充是否应只用 train 统计量
- 指标口径：阈值 0.5 、正负样本比例
- 提交：学号 _ 姓名 _logic.zip 与反思

Agent 会犯错，验收不能外包

幻觉

编造不存在的 API / 结果

→ 要求给出文件、命令、输出

工具错误

命令失败但被忽略

→ 看 exit code 与 stderr

Context rot

关键信息被长上下文淹没

→ 压缩、笔记、只读相关文件

权限越界

写到工作区之外

→ 沙箱、最小权限、人工确认

不可复现

没有 seed / 版本 / 命令

→ 记录环境与完整命令

过度依赖

把“它说完成”当完成

→ 测试、指标、人工复核

Agent proposes, evidence decides.

课堂练习：把实验交给 Agent 之前

01

写一份任务书

用 5 句话写清：目标、输入、约束、输出、验收标准。

02

找三个检查点

在本实验中，哪三件事必须人工确认，不能交给 Agent？

03

加一条验收

给 verify_submission.py 增加一个你能想到的检查，并说明为什么。

10 分钟 · 两人一组 · 分享一个你认为 Agent 最可能骗过你的地方。

带走五句话

01 逻辑回归 = 线性分数 + sigmoid + BCE。

02 数据管线决定上限；形状与数值稳定决定能否跑通。

03 评估看混淆矩阵、 ROC 与阈值，不只看 accuracy。

04 Agent = LLM + tools + loop； workflow 不等于 agent。

05 Prompt < Context < Harness；验收是 Harness 的一部分。

模型能力 × Harness 能力 = 可交付结果

这是教学近似，不是定律；但它解释了为什么“同一个模型，换个环境就完全不同”。

参考资料

01 Russell & Norvig. AIMA, Ch. 2 Intelligent Agents.

02 Wooldridge & Jennings. Intelligent Agents: Theory and Practice, 1995.

03 Anthropic. Building effective agents, 2024.

04 Anthropic. Effective context engineering for AI agents, 2025.

05 Yao et al. ReAct, arXiv:2210.03629, 2022.

06 Yang et al. SWE-agent, arXiv:2405.15793, 2024.

07 OpenAI. Harness engineering, 2026 （页面 403， 标题 / 年份经检索核对）.

08 DeepSeek Harness 官方介绍， 2026.

09 DSH 架构文档： core subsystems （agent-loop / session / tools）.

10 AGENTS.md: a simple open format for guiding coding agents.

11 Schulhoff et al. The Prompt Report, arXiv:2406.06608, 2024.

12 课程《逻辑回归 - 实验手册》§1.6； 本工作区 WORKLOG / ACCEPTANCE.

访问日期： 2026-10-28 。完整链接与“事实 / 教学类比”标注见 design/research/agent_harness_brief.md 。